

平时作业三

平时作业三

问题一

题目

回答

问题二

题目

回答

补充：用小程序做验证

问题三

题目

回答

1. `ori` 使用 0 扩展
2. `lw` / `sw` 使用符号扩展的是地址偏移量
3. 总结

问题一

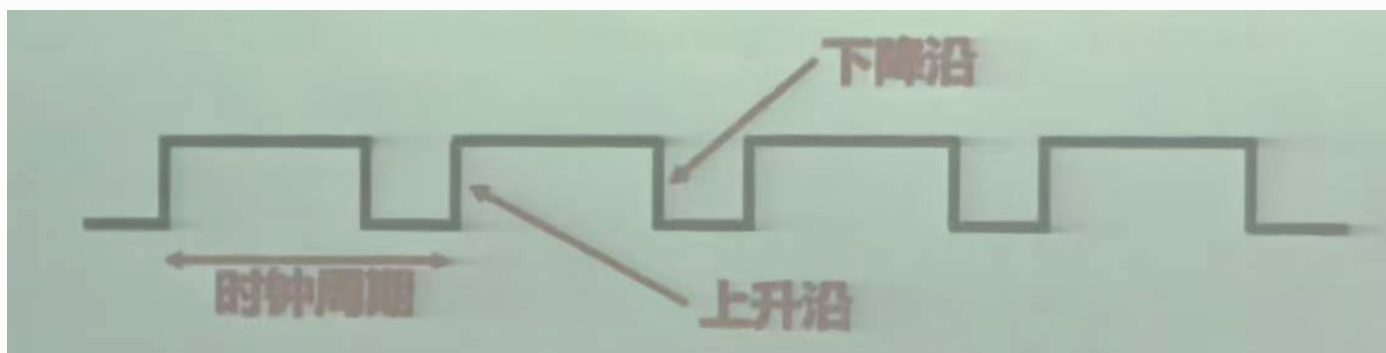
题目

状态（存储）元件的特点：

- 具有存储功能，在时钟控制下，输入被写入电路并保持到下一个时钟到达；
- 输入端状态由时钟决定何时写入，输出端状态可以随时读出。

定时方式：规定信号何时写入状态元件，或何时从状态元件读出。

- 边沿触发 (edge-triggered) 方式：
 - 状态单元中的值只在时钟边沿改变，每个时钟周期改变一次；
 - 上升沿 (rising edge) 触发：在时钟正跳变时进行读/写；
 - 下降沿 (falling edge) 触发：在时钟负跳变时进行读/写。



问题：为什么用时钟边缘（下降沿）做控制信号？

回答

使用时钟边沿作为控制信号，主要是为了使状态元件只在一个短暂且确定的时刻更新，从而将“组合逻辑计算”和“状态写入”明确区分开来。

在同步时序电路中，一个时钟周期内通常包含以下过程：

1. 寄存器输出上一个周期保存的状态；
2. 组合逻辑根据当前状态计算新的结果；
3. 到下一个有效时钟边沿时，寄存器将已经稳定的新结果写入。

如果状态元件在整个高电平或低电平期间都允许写入，就会出现“透明”问题：输入变化可能在同一个时钟电平期间连续通过多个状态元件，造成竞争、冒险，或者导致一次周期内发生不可预期的多次更新。边沿触发可以避免这类问题，使每个状态元件在每个周期只更新一次，也便于建立清晰、可分析的时序约束。

至于选择下降沿而不是上升沿，本质上是一种时序设计约定。只要整个系统保持一致，上升沿和下降沿都可以作为有效边沿。具体采用下降沿，通常有以下考虑：

1. 与其他在上升沿工作的部件错开，减少同一时刻发生读写冲突的可能；
2. 为组合逻辑的传播和稳定留出时间；
3. 保证输入信号在下降沿到来前满足建立时间，在下降沿之后满足保持时间。

因此，下降沿本身并没有特殊性。真正重要的是采用边沿触发，并保证所有相关信号都围绕这个有效边沿满足时序要求。

问题二

题目

反汇编得到的 `outputs` 汇编代码：

```
080483e4: push    %ebp
080483e5: mov     %esp,%ebp
080483e7: sub     $0x18,%esp
080483ea: mov     0x8(%ebp),%eax
080483ed: mov     %eax,0x4(%esp)
080483f1: lea     -0x10(%ebp),%eax
080483f4: mov     %eax,(%esp)
080483f7: call    0x8048330 <__gmon_start__@plt+16>
080483fc: lea     -0x10(%ebp),%eax
080483ff: mov     %eax,0x4(%esp)
08048403: movl    $0x8048500,(%esp)
0804840a: call    0x8048310 <printf@plt>
0804840f: leave
08048410: ret
```

其中，`080483ea` 到 `080483f4` 是将两个 `strcpy` 的参数入栈，`080483fc` 到 `08048403` 是将 `printf` 的两个参数入栈。

问题：解释 `__gmon_start__` 的含义。

回答

`__gmon_start__` 是 GNU 工具链和 glibc 中与性能分析 (profiling) 相关的运行时符号，名称中的 `gmon` 来自 GNU monitor。当程序使用 `gprof` 等工具进行性能统计时，运行时初始化代码可能会通过该符号启动相应的采样或统计机制。

即使普通程序没有显式调用 `__gmon_start__`，反汇编结果中也可能出现该符号。这是因为它与 ELF 动态链接、运行时初始化以及 PLT/GOT 机制有关。它通常以弱符号形式出现：如果程序没有启用 profiling，相应实现可以不存在，运行时也不会真正执行额外的 profiling 初始化。

还需要注意，`@plt` 和 `+16` 都不是符号名本身的一部分。以题中这一行为例：

```
080483f7: call    0x8048330 <__gmon_start__@plt+16>
```

可以将其拆开理解：

部分	含义
<code>__gmon_start__</code>	反汇编器识别到的一个已知符号名
<code>@plt</code>	该符号对应或靠近 PLT (Procedure Linkage Table) 入口
<code>+16</code>	当前地址相对于该符号地址偏移 16 字节

因此，`<__gmon_start__@plt+16>` 并不等同于“调用 `__gmon_start__`”。更准确地说，它表示这个调用地址位于 `__gmon_start__@plt` 之后 16 字节处。

题目已经指出，`080483ea` 到 `080483f4` 是在准备 `strcpy` 的两个参数，因此 `080483f7` 这一条 `call` 的实际语义应当是调用 `strcpy` 的 PLT 入口。反汇编器之所以将其显示为 `<__gmon_start__@plt+16>`，是因为 PLT 表中每个入口通常占固定大小，例如 32 位 x86 中常见为 16 字节；当符号信息不完整或反汇编器只能采用相邻符号进行标注时，就可能用“最近的已知符号 + 偏移量”的形式表示地址。

简言之：

- `__gmon_start__` 是与 GNU profiling 初始化相关的运行时符号；
- `@plt+16` 表示 PLT 入口附近的偏移地址，不是符号名的一部分；

3. 本题中的 `<__gmon_start__@plt+16>` 应理解为反汇编器的地址标注方式，实际调用目标应是 `strcpy@plt`。

补充：用小程序做验证

为了验证上述解释，可以编写一个与题目逻辑相近的小程序。验证重点不是证明源程序调用了 `__gmon_start__`，而是观察以下现象：

1. `strcpy` 这类库函数会通过 PLT 调用；
2. `__gmon_start__` 会作为动态链接/profiling 相关符号出现在符号表中；
3. `__gmon_start__` 是弱符号，而不是这段 C 代码主动调用的普通函数。

验证程序如下：

```
#include <stdio.h>
#include <string.h>

void outputs(const char *input) {
    char buf[16];

    strcpy(buf, input);
    printf("%s\n", buf);
}

int main(int argc, char **argv) {
    if (argc > 1) {
        outputs(argv[1]);
    }

    return 0;
}
```

在 Linux 32 位环境下编译：

```
gcc -m32 -O0 -fno-stack-protector -no-pie -o q2_verify
q2_verify.c
```

然后进行反汇编：

```
objdump -d -M intel q2_verify
```

得到的 `outputs` 函数关键部分如下：

```
0804843b <outputs>:
804843b: 55                push    ebp
804843c: 89 e5            mov     ebp,esp
804843e: 83 ec 18        sub     esp,0x18
8048441: 83 ec 08        sub     esp,0x8
8048444: ff 75 08        push   DWORD PTR [ebp+0x8]
8048447: 8d 45 e8        lea     eax,[ebp-0x18]
804844a: 50             push   eax
804844b: e8 b0 fe ff ff  call   8048300 <strcpy@plt>
8048450: 83 c4 10        add     esp,0x10
8048453: 83 ec 0c        sub     esp,0xc
8048456: 8d 45 e8        lea     eax,[ebp-0x18]
8048459: 50             push   eax
804845a: e8 b1 fe ff ff  call   8048310 <puts@plt>
804845f: 83 c4 10        add     esp,0x10
8048463: c9             leave
8048464: c3             ret
```

这段反汇编可以说明：

1. `8048444` 和 `804844a` 两条 `push` 指令是在准备 `strcpy` 的两个参数；
2. 随后的 `804844b` 调用的是 `strcpy@plt`；
3. `printf("%s\n", buf)` 被编译器简化成了等价的 `puts(buf)`，所以后面看到的是 `puts@plt`，而不是 `printf@plt`。

这与题目中的现象一致：题目里 `080483ea` 到 `080483f4` 先准备了 `strcpy` 的两个参数，因此紧接着的 `call` 从语义上应当是调用 `strcpy`。如果反汇编器将该地址显示为 `<__gmon_start__@plt+16>`，它只是用“已知符号 + 偏移量”的形式标注地址，不能据此认为程序真的调用了 `__gmon_start__`。

如果某些反汇编输出把第一条 `call` 标成：

```
call 8048330 <__gmon_start__@plt+16>
```

也不能据此认为程序调用的是 `__gmon_start__`。因为 `+16` 表示当前地址位于 `__gmon_start__@plt` 之后 16 字节处；在 32 位 x86 的 PLT 中，一个 PLT 入口常常正好占 16 字节，所以这个地址很可能已经是下一个 PLT 入口。

再查看动态符号表：

```
nm -D q2_verify
```

实际输出为：

```
      w __gmon_start__
0804851c R _IO_stdin_used
      U __libc_start_main
      U puts
      U strcpy
```

这个输出可以这样解释：

符号	含义
<code>w __gmon_start__</code>	小写 <code>w</code> 表示 weak symbol，即弱符号。它可以没有具体定义，不影响程序正常链接和运行。
<code>U strcpy</code>	<code>U</code> 表示 undefined，即未定义符号，需要运行时从动态库中解析；这说明程序确实调用了库函数 <code>strcpy</code> 。
<code>U puts</code>	这里出现 <code>puts</code> 而不是 <code>printf</code> ，是因为编译器把 <code>printf("%s\n", buf)</code> 简化成了等价的 <code>puts(buf)</code> 。
<code>U __libc_start_main</code>	C 程序启动时由运行时库使用的入口相关符号。
<code>R _IO_stdin_used</code>	位于只读数据区的 glibc 兼容性标记，和本题核心无关。

因此，这个小程序验证了本题结论： `__gmon_start__` 是运行时 profiling/动态链接相关的弱符号； `strcpy` 才是 `outputs` 函数中真实调用的库函数。题目中的 `<__gmon_start__@plt+16>` 应理解为反汇编器的地址标注方式，而不是源程序主动调用 `__gmon_start__`。

问题三

题目

OR Immediate 指令格式：

```
ori rt, rs, imm16
```

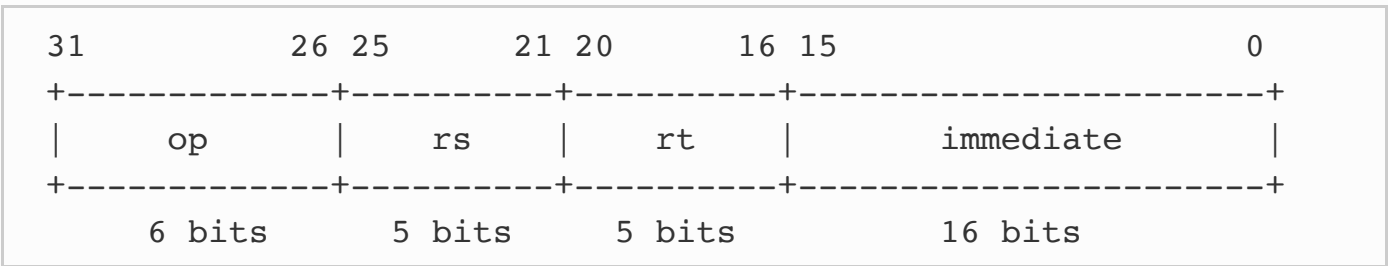
Load Word 指令格式：

```
lw rt, rs, imm16
```

Store Word 指令格式：

```
sw rt, rs, imm16
```

I 型指令格式：



问题：为什么 `ori` 指令是 0 扩展，而 `load` 是符号扩展？

回答

MIPS 的 I 型指令都包含 16 位立即数字段，但立即数的扩展方式取决于指令语义。也就是说，不能仅仅因为指令格式相同，就认为所有立即数都应该采用同一种扩展方式。

1. `ori` 使用 0 扩展

`ori rt, rs, imm16` 是按位逻辑或运算。这里的 `imm16` 更适合被理解为一个 16 位无符号位模式，而不是一个带有正负含义的整数。

因此，执行时会先将立即数 0 扩展成 32 位：

```
imm16 = 0x8001
0 扩展后: 0x00008001
```

然后再与 `rs` 做按位或运算。

这种设计还有一个重要用途：配合 `lui` 构造 32 位常数。例如：

```
lui $t0, 0x1234      # $t0 = 0x12340000
ori $t0, $t0, 0xffff # $t0 = 0x1234ffff
```

如果 `ori` 对 `0xffff` 进行符号扩展，就会得到 `0xffffffff`，再与任何数做或运算都会把高 16 位也置为 1，结果会变成 `0xffffffff`，从而破坏构造常数的目的。因此，`ori` 必须使用 0 扩展，才能保持立即数作为“位模式”的含义。

2. `lw` / `sw` 使用符号扩展的是地址偏移量

对 `lw` 和 `sw` 来说，16 位立即数不是普通的逻辑位模式，而是访存地址中的偏移量。访存指令通常写成：

```
lw rt, imm16(rs)
sw rt, imm16(rs)
```

有效地址计算为：

```
effective_address = rs + sign_extend(imm16)
```

这里需要特别区分两件事：`lw` 加载的是完整的 32 位 word，加载结果本身不需要再做符号扩展；被符号扩展的是指令中的 16 位地址偏移量。对于 `lb/lbu`、`lh/ldu` 这类加载较小数据宽度的指令，才会涉及“加载出的数据”是符号扩展还是 0 扩展的问题。

地址偏移量需要能够表示正数，也需要能够表示负数。例如访问栈帧时，经常需要相对于 `$sp` 或 `$fp` 使用负偏移：

```
lw $t0, -4($sp)
```

`-4` 的 16 位补码表示是 `0xffffc`。如果进行符号扩展：

```
0xffffc -> 0xfffffffcc = -4
```

地址计算结果正确，表示 `$sp - 4`。

如果错误地使用 0 扩展：

```
0xffffc -> 0x0000fffcc = 65532
```

地址就会变成 `$sp + 65532`，语义完全错误。

因此，`load/store` 指令中的立即数需要进行符号扩展，是因为它表示相对于基址寄存器的有符号位移，从而允许访问基址附近正负两个方向的内存地址。

3. 总结

`ori` 和 `lw/sw` 的立即数字段都是 16 位，但用途不同：

指令	立即数含义	扩展方式	原因
<code>ori</code>	逻辑运算中的位模式	0 扩展	保持低 16 位位模式，不让符号位影响高 16 位
<code>lw/sw</code>	基址寻址中的地址偏移量	符号扩展	偏移量需要能够表示正负方向，尤其是栈帧中的负偏移

所以，`ori` 的 0 扩展是为了符合逻辑运算和构造常数的需求；`load/store` 的符号扩展是为了符合地址计算中“有符号偏移量”的需求。